SCoPe: Trust-Region Latent Planning with JEPA World Models

22AIE401 Reinforcement Learning

Aayush Shenday
AM.SC.U4AIE23004
Amrita School of Computing, Amritapuri

Rajashekar Kasarla
AM.SC.U4AIE23041
Amrita School of Computing, Amritapuri

***Abstract*—Planners built on learned world models choose the action sequence whose *imagined* outcome looks best, but they trust every imagined future equally, including the ones the model gets badly wrong. We present SCoPe, a decision-time trust check for latent model-predictive control (MPC) with a frozen joint-embedding predictive (JEPA) world model: every imagined step of every candidate plan receives a trust score, and candidates that leave the region where the model is reliable are rejected before a plan is chosen. Building on DINO-WM, we reproduce its planning results on an 8 GB laptop GPU, made feasible by runtime optimisations that cut rollout time 3.8× without changing any decision of the cross-entropy-method (CEM) planner: Push-T reaches 0.900 over 3 seeds (published 0.90), Wall 0.947 over 3 seeds (0.96) and PointMaze 1.000 (0.98). We then measure how well five training-free trust scores detect real prediction errors, on 76,800 imagined steps that we verify against the simulator over 3 seeds. A Monte-Carlo-dropout score identifies the steps with the largest real error at AUROC 0.86 (95% CI 0.84–0.88), above a random score (0.50) and a strong trivial baseline (0.79) on every seed. A replay of the trust gate on recorded candidates shows that filtering the final candidate set barely changes success, even with an oracle gate, which identifies where the gate must act: inside the replanning loop rather than as a one-shot filter.**

***Index Terms*—world models, model-based reinforcement learning, model-predictive control, JEPA, uncertainty estimation**

I. Introduction

A robot can plan by imagining the consequences of its actions with a learned world model and executing the action sequence whose imagined result is closest to its goal. Joint-embedding predictive architectures (JEPA) [4], [5] make this practical from images: instead of predicting pixels, they predict the future in the embedding space of a pretrained encoder. DINO-WM [1] combines a frozen DINOv2 encoder [3] with a learned latent predictor and plans with the cross-entropy method (CEM) inside MPC, reaching 90% success on the Push-T manipulation task without any reward or task-specific training; V-JEPA 2-AC [2] applies the same idea to real robot arms.

These planners have a blind spot: the planner selects the candidate whose imagined future scores best and never asks whether that imagination is reliable. Fig. 1 shows a case from our own runs: the model imagines the T-shaped block rotating onto the goal, the planner commits to the plan, and in the simulator the block barely moves. Prediction errors also grow with the horizon, and simply penalising uncertainty can backfire [18].

Research question. Can a score computed only from the imagined latent states tell which imagined steps are wrong, reliably enough to decide which candidate plans the planner may act on?

Novel contributions.

1) *Trust-region planning for JEPA world models.* SCoPe formulates latent MPC as cost minimisation under a per-step trust constraint (Eq. 1) and enforces it with a gate inside CEM, with a fallback ladder for when too few candidates pass. The world model is never retrained, so the method applies to any frozen backbone.
2) *An imagined-against-real evaluation protocol.* Every imagined step of CEM's candidates is labelled with its real latent error by executing the candidate in the simulator from the same start state, and the label is defined per step index so that the growth of error with the horizon cannot inflate a score.
3) *A training-free trust score that works.* Of five scores, MC-dropout on the frozen predictor detects the most wrong imagined steps at AUROC 0.86, above a random score and above a strong trivial baseline on all 3 seeds. We also show that the planner's own imagined cost carries no information about prediction error.
4) *Where a trust gate must act.* A replay with random, trust-score and oracle gates shows that rejecting candidates only at the final selection barely changes success, even when the true error is known, so the gate has to change the search across replanning rounds.
5) *An efficient, verified reproduction.* DINO-WM planning on three tasks on an 8 GB laptop GPU, 3.8× faster than the unmodified code, with the planner's choices verified to be unchanged.

II. Background and Related Work

Uncertainty in model-based RL. PETS [6] plans with an ensemble of probabilistic models; MBPO [8] limits model rollouts to short horizons; MOPO [9] and MOReL [10] penalise or halt rollouts in uncertain regions for offline RL; MACURA [11] adapts the rollout length to model uncertainty.

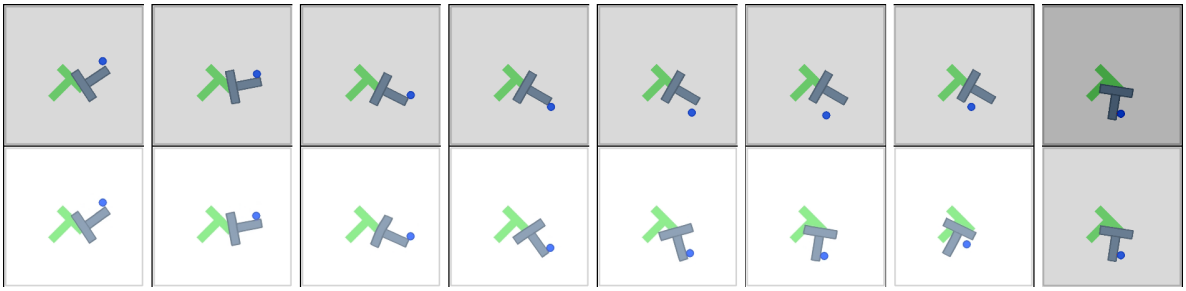


Fig. 1. A logged failure (Push-T, seed 101, episode 0, simulator steps 0–25 and the goal). Top: what happened in the simulator. Bottom: what DINO-WM imagined for the same actions. The imagined block reaches the goal pose; the real one barely moves, and the episode is never solved. SCoPe is designed to flag such imagined steps before a plan is chosen.

All of them distrust the model *while training a policy*, mostly on low-dimensional states.

Latent world models. PlaNet [7], DreamerV3 [12] and TD-MPC2 [13] learn latent dynamics with a reward; JEPA-style planners such as DINO-WM [1], PLDM [14], HWM [15] and V-JEPA 2-AC [2] are reward-free and plan towards a goal image, but they trust every imagined rollout equally. Latent safety filters [16] reject actions in latent space for safety constraints rather than prediction errors.

Known pitfalls. Ensemble disagreement can miss compounding error in latent models [17]; uncertainty penalties raised collisions in a navigation study [18]; and diagnostics of JEPA planners point to action-consistency [19], representation geometry [20] and stage-wise reliability [21]. We therefore test every trust score against the real error rather than assuming it works, with deep ensembles [22] and MC-dropout [23] as the classical candidates.

Difference from prior work. SCoPe changes one step of the planning loop: it scores each imagined step of each CEM candidate at decision time, with a frozen backbone, and rejects untrusted candidates before a plan is chosen.

III. Problem Formulation

We model goal-reaching as a finite-horizon MDP without a hand-designed reward. The observation o_t is a 224×224 image and the agent's proprioceptive state p_t. The frozen encoder maps it to the latent state $z_t = E(o_t) \in \mathbb{R}^{196\times384}$ (DINOv2 ViT-S/14 patch tokens). An action is a 2-D target displacement; actions are grouped in chunks of five simulator steps, $a_k \in \mathbb{R}^{10}$. The predictor f_θ imagines $\hat{z}_{k+1} = f_\theta(\hat{z}_{k-2:k}, a_k)$ from a three-frame history. Real transitions come from the simulator. An episode succeeds when the environment's own criterion holds at the end (for Push-T, block and agent within 20 units and $\pi/9$ of the goal pose).

Given the goal latent z_g, DINO-WM plans $H = 5$ chunks ahead by minimising $C(a_{1:H}) = \|\hat{z}_H - z_g\|^2 + \alpha\,\|\hat{p}_H - p_g\|^2$. SCoPe adds a constraint, not a reward:

$$\min_{a_{1:H}} C(a_{1:H}) \quad \text{s.t.} \quad u(\hat{z}_k) \le \tau \ \text{ for all } k = 1,\dots,H, \qquad (1)$$

where u is a trust score (higher means less trusted) and τ is a threshold. This is a sequential decision problem rather than a supervised one because a push takes effect several steps later and success is judged at the end; we use CEM-MPC rather than PPO or SAC because the data are offline and reward-free, and because CEM keeps an explicit set of candidates that a gate can filter.

IV. Method: SCoPe

A. Architecture

Fig. 2 shows the system. At each decision the simulator returns an observation, the encoder maps it to a latent state, CEM samples candidate action sequences, and the predictor imagines each of them. The trust module scores every imagined step, the gate removes candidates with an untrusted step, and only accepted candidates compete on cost. For evaluation, the imagined latents are logged next to the encodings of what really happened. The world model is never retrained. Backbones and scorers plug in through two small interfaces, so V-JEPA 2-AC can replace DINO-WM without changes to the planner.

B. Trust scores

All scores are computed from the imagination alone, need no training, and are larger for less trusted steps:

- **MC-dropout** [23]: variance of $\hat{z}_k$ over four predictor passes with its dropout (0.1, as trained) on.
- **Action perturbation**: divergence of the imagined rollout when the actions receive Gaussian noise ($\sigma = 0.1$).
- **Image perturbation**, in the spirit of [19]: divergence when the start image receives noise ($\sigma = 0.1$).
- **Distance to training data**: mean distance to the five nearest encoded training frames (random projection to 512 dimensions).
- **Imagined cost**: the plan's own predicted distance to the goal.
- Baselines: **distance moved**, $\|\hat{z}_k - z_0\|$, a trivial score, and **random**.

C. Gated CEM-MPC with a fallback ladder

Algorithm 1 gives the decision loop. Imagined latents are never used as the start of the next plan: after executing the accepted actions, the agent observes again. When too few candidates pass, the ladder widens the search, then shortens the horizon, then takes a safe no-op, and logs which rung fired.

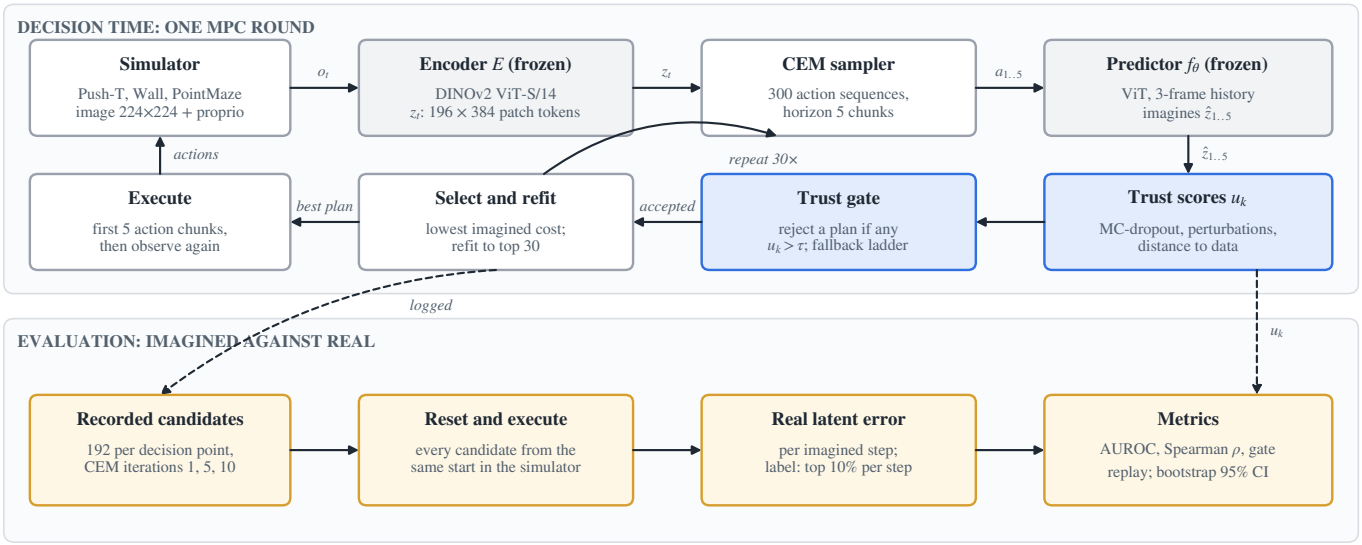


Fig. 2. SCoPe architecture. Top: one MPC round at decision time. The frozen DINO-WM encoder and predictor (grey) imagine every CEM candidate; the trust module (blue) scores each imagined step and the gate removes untrusted candidates before selection. Bottom: the evaluation path, which executes recorded candidates in the simulator and compares the imagined latents with the real ones.

Algorithm 1 SCoPe decision step (one MPC round)

observe o_t; $z_t \leftarrow E(o_t)$
for CEM iteration $i = 1, \ldots, I$ **do**
  sample N action sequences $a^{(n)}_{1:H} \sim \mathcal{N}(\mu, \Sigma)$
  imagine $\hat{z}^{(n)}_{1:H}$ with f_θ; score $u^{(n)}_k$ for every step
  accepted $\mathcal{A} \leftarrow \{n : \max_k u^{(n)}_k \le \tau\}$
  if $|\mathcal{A}| < K$ **then**
    fallback: resample wider $\rightarrow$ $H \leftarrow H/2$ $\rightarrow$ no-op (logged)
  end if
  refit (μ, Σ) to the K lowest-cost candidates in $\mathcal{A}$
end for
execute the first chunks of the best accepted plan; return to line 1

D. Evaluation questions

Q1: Can DINO-WM planning be reproduced on a single 8 GB GPU without changing the planner's decisions? **Q2:** Does a trust score rank imagined steps with large real latent error above the others (AUROC, Spearman ρ), and does it beat both a random and a trivial baseline? We require non-overlapping 95% intervals. **Q3:** When the gate filters the candidate set, does the chosen plan succeed more often than without a gate, and how close does it come to an oracle gate that knows the true error?

V. Experimental Setup

Tasks and data. Push-T [24] (push a T-shaped block onto a target), Wall (pass through a door) and PointMaze [25], with the datasets and released checkpoints of DINO-WM [26]. Push-T goals are the real state 25 steps later on a held-out trajectory; Wall and PointMaze goals are sampled reachable states.

Planner. Upstream settings: 300 samples, top 30, horizon 5, 30 CEM iterations on Push-T and 10 on Wall and PointMaze, five chunks executed per MPC round, at most three rounds (the paper does not state its replanning budget, so we report every round). 50 start/goal pairs per seed; seeds 99, 100 and 101. We read the paper's CEM row as our round 1 (one plan, executed open loop) and its MPC row as success after replanning.

Hardware and runtime optimisations. One RTX 5060 laptop GPU (8 GB), WSL2, PyTorch 2.7.1 with CUDA 12.8. Unmodified, a 300-candidate rollout took 26.2 s and did not fit in memory at full settings. We fused attention, encoded the shared observation once instead of 300 times, chunked rollouts, disabled autograd where unused, and enabled TF32 matrix multiplication: 6.9 s per rollout (3.8×). Against the unmodified model, CEM's top-30 set and its update were identical at three sampling spreads; bf16 was faster but changed 17% of the elites at small spreads and was therefore not used.

Trust-score evaluation. On 80 Push-T decision points over 3 seeds (40, 20 and 20), CEM ran 10 iterations; we recorded 192 of the candidates it evaluated per decision point (48 from iteration 1, 48 from iteration 5 and 96 from iteration 10, so the set spans bad to good plans), imagined each, executed each from the same start state in the simulator, and encoded the real frames at steps $5, 10, \ldots, 25$. This gives 76,800 imagined steps with a real latent next to them. The real error of step k is the mean squared difference of the imagined and real patch tokens; a step is *unreliable* if its error is in the top 10% *for its step index*. AUROC and Spearman ρ are computed per step and averaged; 95% intervals come from 500 bootstrap resamples over decision points.

Gate analysis. Among each decision point's recorded can-

TABLE I
PLANNING SUCCESS OVER 50 START/GOAL PAIRS PER SEED (MEAN ± SD OVER SEEDS).

Task	Seeds	One plan (round 1)	With replanning	Paper [1]
Push-T	3	0.87 ± 0.05	0.90 ± 0.05 (round 3)	0.86 / 0.90
Wall	3	0.52 ± 0.02	0.95 ± 0.02 (round 3)	0.74 / 0.96
PointMaze	1	0.86	1.00 (round 2)	0.80 / 0.98

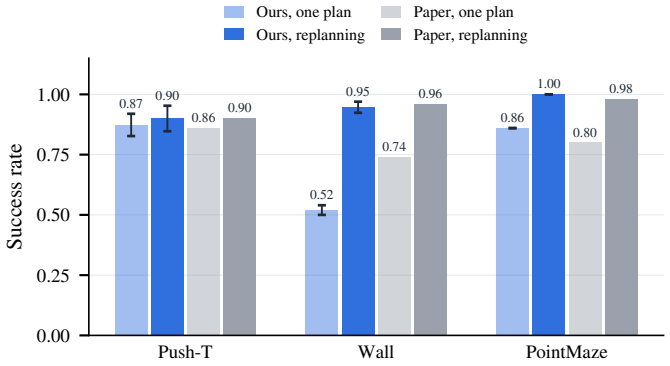


Fig. 3. Planning success after one plan and after replanning: ours (blue, mean ± SD over seeds) against the published DINO-WM results (grey).

didates, plain selection takes the lowest imagined cost; a gate first rejects the 30% least trusted candidates (by their worst step) and then takes the lowest imagined cost. The outcome is whether the chosen candidate succeeds in the simulator. We compare every trust score with a random gate and an oracle gate that rejects by the true error.

VI. RESULTS

A. Q1: Reproduction of DINO-WM

Table I and Fig. 3 summarise the reproduction. On Push-T we obtain 0.873 ± 0.046 after one plan and 0.900 ± 0.053 after replanning (per seed 0.92 / 0.94 / 0.84), against the published 0.86 and 0.90; at 50 pairs the binomial standard error is about 0.04, so both published values lie within the seed spread. PointMaze reaches 1.000 after round 2, above the published 0.98. Wall reaches 0.947 after round 3 (per seed 0.92 / 0.96 / 0.96; published 0.96). Its single-plan result, 0.520 ± 0.020, is consistently below the published 0.74; the most likely cause is that the upstream Wall setting stops CEM after 10 iterations. Replanning brings Wall to the published level. Fig. 4 shows that on Push-T the success of CEM's current plan rises steeply in the first ten iterations and plateaus after about twenty.

B. Q2: Do trust scores predict real error?

Table II and Fig. 5 give the answer. MC-dropout ranks the unreliable steps highest, with AUROC 0.86 (95% CI 0.84–0.88; per seed 0.87, 0.84, 0.88), and its whole interval lies above the trivial baseline (0.79, 0.76–0.82) and the random score (0.50). It is the only score that meets the criterion of Q2, and it does so on every seed. Action perturbation (0.66)

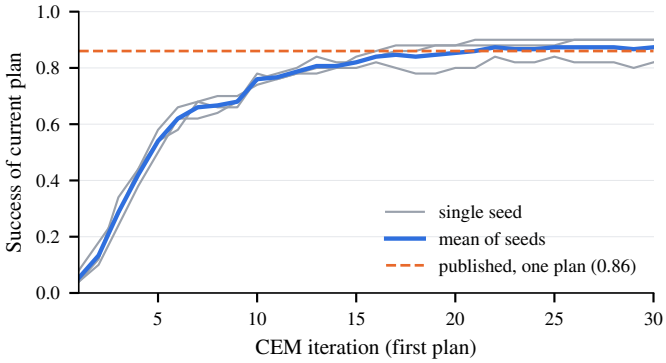


Fig. 4. Push-T, first plan: success rate of CEM's current mean plan after each iteration (evaluation only; the planner never sees it). Grey: seeds; blue: mean; dashed: published single-plan result.

TABLE II
AUROC FOR THE TOP-10% REAL LATENT ERROR (PER STEP), 80 PUSH-T DECISION POINTS, 3 SEEDS.

Trust score	AUROC	95% CI	Spearman ρ
MC-dropout (4 passes)	**0.863**	**0.84–0.88**	0.65
Action perturbation	0.657	0.64–0.67	0.27
Distance to training data	0.619	0.56–0.66	0.12
Imagined cost	0.511	0.47–0.55	0.04
Image perturbation	0.496	0.44–0.55	−0.17
MC-dropout + distance (rank avg.)	0.851	0.83–0.87	0.64
Distance moved (baseline)	0.790	0.76–0.82	0.51
Random (baseline)	0.500	0.49–0.51	0.00

and distance to the training data (0.62) carry some signal but less than the trivial baseline. The plan's own imagined cost carries none (0.51): a confident planner is no evidence of a correct imagination. Image perturbation is at chance (0.50), most likely because small noise on the input image is absorbed by the frozen encoder, so it probes the encoder rather than the predictor. The strength of the trivial baseline is itself informative: plans that change the scene a lot are the ones the model gets wrong, so every trust score must be compared with it, not only with chance. A rank average of MC-dropout and the trivial baseline, fixed in advance, does not improve on MC-dropout alone.

C. Q3: Gate analysis

Table III shows the result. The plan chosen without a gate succeeds in 71.3% of decision points, and 70% with the MC-dropout gate; one decision point is worth 1.3%, so the differences between gates are within noise. The key row is the oracle: even rejecting by the *true* error barely changes success (72.5% against 71.3%). Among the recorded candidates, the one with the lowest imagined cost is rarely a high-error one, so a filter applied once, at the final selection, has little to change. Detecting error (Q2) is therefore necessary but not sufficient: the benefit of a trust gate has to come from changing which candidates CEM refits to in every iteration and round, as in Algorithm 1, and from conditions where the model is wrong more often.

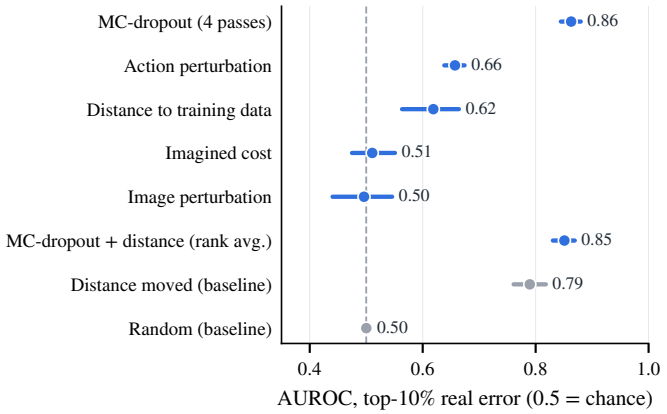


Fig. 5. AUROC with 95% bootstrap intervals over decision points. Blue: trust scores; grey: baselines; dashed: chance.

TABLE III
GATE ANALYSIS ON RECORDED CANDIDATES (REJECT 30%, THEN LOWEST IMAGINED COST), 80 DECISION POINTS.

Selection among recorded candidates	Chosen plan succeeds
Plain (lowest imagined cost)	71.3%
Oracle gate (true error)	72.5%
Random gate	70%
MC-dropout gate	70%
Action-perturbation gate	75%
Distance-to-data gate	68.8%
Distance-moved gate	67.5%

VII. Discussion and Limitations

The results support the central premise of SCoPe: the frozen world model's own dropout already carries a reliable signal of where its imagination is wrong, at no training cost and four extra predictor passes per candidate. They also show two things that a gate design must respect. First, a trivial score based on how far the imagined state moves is a strong baseline (AUROC 0.79), so a trust score that only beats chance proves little. Second, filtering the final candidates is not enough, even with perfect knowledge of the error.

The trust scores and the gate were evaluated on Push-T, with candidates taken from CEM's first plan. All tasks are simulated and 2-D; real-world use would require a backbone such as V-JEPA 2-AC with robot data. Planning takes minutes per decision on our GPU, so the system is not real-time. Upstream MPC stops solved episodes using the simulator's success flag; we keep this identical for every method.

VIII. Conclusion

We built SCoPe, a trust check for latent planning with frozen JEPA world models, and evaluated it on top of a verified reproduction of DINO-WM. The reproduction matches the published success on Push-T (0.900) and Wall (0.947) and exceeds it on PointMaze (1.000), on an 8 GB GPU and 3.8× faster than the unmodified code. Across 76,800 imagined steps checked against the simulator, an untrained MC-dropout score detects the most wrong predictions at AUROC 0.86, clearly above random and above a strong trivial baseline on every seed, while the planner's own cost carries no such signal. The gate analysis shows that this signal must be used inside the search, not only at the final choice.

Future work. Applying the gate inside every CEM iteration over several replanning rounds on all three tasks, an ensemble score and calibration of τ, evaluation under visual shifts, and transfer to V-JEPA 2-AC.

Reproducibility

Code, run scripts and the report sources are in the project repositories (scope-planning, scope-project, scope-mid-review). Every number in Tables I–III and in the text is generated from the run logs by `report/make_assets.py`.

References

[1] G. Zhou, H. Pan, Y. LeCun, L. Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. ICML 2025, PMLR 267:79115–79135. https://arxiv.org/abs/2411.04983

[2] M. Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025. https://arxiv.org/abs/2506.09985

[3] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024. https://arxiv.org/abs/2304.07193

[4] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). ICCV 2023. https://arxiv.org/abs/2301.08243

[5] Y. LeCun. A Path Towards Autonomous Machine Intelligence. Position paper, v0.9.2, 2022 (not peer-reviewed). https://openreview.net/forum?id=BZ5a1r-kVsf

[6] K. Chua, R. Calandra, R. McAllister, S. Levine. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS). NeurIPS 2018. https://arxiv.org/abs/1805.12114

[7] D. Hafner et al. Learning Latent Dynamics for Planning from Pixels (PlaNet). ICML 2019, PMLR 97:2555–2565. https://arxiv.org/abs/1811.04551

[8] M. Janner, J. Fu, M. Zhang, S. Levine. When to Trust Your Model: Model-Based Policy Optimization (MBPO). NeurIPS 2019. https://arxiv.org/abs/1906.08253

[9] T. Yu et al. MOPO: Model-based Offline Policy Optimization. NeurIPS 2020. https://arxiv.org/abs/2005.13239

[10] R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. MOReL: Model-Based Offline Reinforcement Learning. NeurIPS 2020. https://arxiv.org/abs/2005.05951

[11] B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe. Trust the Model Where It Trusts Itself: Model-Based Actor-Critic with Uncertainty-Aware Rollout Adaption (MACURA). arXiv:2405.19014, 2024. https://arxiv.org/abs/2405.19014

[12] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering Diverse Control Tasks through World Models (DreamerV3). Nature 640:647–653, 2025. https://doi.org/10.1038/s41586-025-08744-2

[13] N. Hansen, H. Su, X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. ICLR 2024. https://arxiv.org/abs/2310.16828

[14] V. Sobal et al. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models (PLDM). NeurIPS 2025. https://arxiv.org/abs/2502.14819

[15] W. Zhang et al. Hierarchical Planning with Latent World Models (HWM). arXiv:2604.03208, 2026. https://arxiv.org/abs/2604.03208

[16] J. Seo, K. Nakamura, A. Bajcsy. Uncertainty-aware Latent Safety Filters for Avoiding Out-of-Distribution Failures. CoRL 2025. https://arxiv.org/abs/2505.00779

[17] J. Berger, B. Frauenknecht, S. Trimpe, B. Leibe. Biased Dreams: Limitations to Epistemic Uncertainty Quantification in Latent Dynamics Models. RLC 2026. https://arxiv.org/abs/2604.25416

[18] Z. Wang. Learning from World Feedback: Why Model Uncertainty Fails as a Risk Signal in Model-Based RL. ICML 2026 Workshop on RL from X Feedback. https://arxiv.org/abs/2607.16591

[19] G. An et al. Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency (ACPC). arXiv:2608.12939, 2026. https://arxiv.org/abs/2608.12939

[20] J. Hu et al. Representation Geometry Matters for Planning with JEPA World Models. arXiv:2608.16287, 2026. https://arxiv.org/abs/2608.16287

[21] G. Lee, B.-T. Zhang. Beyond Task Success: Stage-Wise Reliability of World Model Planning under Sensing Degradation. arXiv:2609.07126, 2026. https://arxiv.org/abs/2609.07126

[22] B. Lakshminarayanan, A. Pritzel, C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NIPS 2017. https://arxiv.org/abs/1612.01474

[23] Y. Gal, Z. Ghahramani. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. ICML 2016. https://arxiv.org/abs/1506.02142

[24] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023 (origin of the Push-T task used by DINO-WM). https://arxiv.org/abs/2303.04137

[25] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. arXiv:2004.07219, 2020 (PointMaze). https://arxiv.org/abs/2004.07219

[26] DINO-WM code (MIT licence), pinned at commit 0a9492f; datasets and checkpoints on OSF (project bmw48). https://github.com/gaoyuezhou/dino_wm